Python Strings

Part 2: Working with Strings

Video #4.2 | PyMaster India

String Methods · String Formatting · Raw Strings · Practice

■■ String Methods	**✍■ String Formatting**	**■ Raw Strings**
20+ essential built-in methods	f-strings, .format(), % operator	Backslashes without escaping
■ Search Methods	**✂■ Split & Join**	**■ Practice**
find, index, count	The most used method pair	30 exercises with solutions

■ Prerequisites: Video #4.1 (String Creation, Indexing, Slicing)

Rajeev Shivah Full-Stack Developer & Data Scientist | codeWithShivah | learn.rajeevshivah.me

© Rajeev Shivah | codeWithShivah | Not for redistribution

Table of Contents

1.	String Methods — Overview	3
2.	Case Methods	3
3.	Strip Methods	4
4.	Search Methods	4
5.	Replace, Split & Join	5
6.	Check Methods (is... methods)	6
7.	Padding & Alignment Methods	7
8.	String Formatting	8
9.	Raw Strings	10
10.	Method Quick Reference Table	11
11.	Practice Exercises	12
12.	Quick Reference Cheat Sheet	14

© Rajeev Shivah | codeWithShivah | Not for redistribution

1. String Methods — Overview

Python's built-in toolkit for working with text

Python strings come with **40+ built-in methods**. You call them using **dot notation**: **string.method()**. Every string method returns a **new string** — the original is never changed (remember: strings are immutable).

```
s = "hello world"
# Dot notation — call method on the string
result = s.upper()
print(result) # HELLO WORLD
print(s) # hello world ← original unchanged!
# You can chain methods
print(s.strip().title()) # Hello World
```

■ Key Rule

String methods NEVER modify the original string.

Always capture the result: result = s.upper() not just s.upper()

Methods can be chained: s.strip().lower().replace('a','b')

2. Case Methods

Changing the case of characters

```
s = "hello WORLD"
print(s.upper()) # HELLO WORLD — all uppercase
print(s.lower()) # hello world — all lowercase
print(s.title()) # Hello World — first letter of each word
print(s.capitalize()) # Hello world — only very first letter
print(s.swapcase()) # HELLO world — flip every case
```

Method	Result	Description
upper()	HELLO WORLD	All characters uppercase
lower()	hello world	All characters lowercase
title()	Hello World	First letter of every word uppercase
capitalize()	Hello world	Only very first character uppercase
swapcase()	HELLO world	Uppercase ↔ lowercase for every char

3. Strip Methods

Removing unwanted whitespace or characters

```
s = ' hello world '
print(s.strip()) # "hello world" — removes both sides
print(s.lstrip()) # "hello world " — removes left only
print(s.rstrip()) # " hello world" — removes right only
# Strip specific characters (not just spaces)
s2 = "***hello***"
print(s2.strip("*")) # "hello"
s3 = "...hello..."
print(s3.strip(".")) # "hello"
```

■ Real World Use

strip() is almost always used when reading user input or file data,

where accidental leading/trailing spaces cause comparison bugs.

Example: if input().strip() == "yes": ← always strip user input first

4. Search Methods

Finding substrings within a string

```
s = "Python is fun and Python is powerful"
# find() — index of first match, returns -1 if not found
print(s.find("Python")) # 0
print(s.find("Python", 5)) # 18 ← search from index 5 onward
print(s.find("Java")) # -1 ← not found, NO error
# index() — same as find() but raises ValueError if not found
print(s.index("is")) # 7
# s.index("Java") # ■ ValueError: substring not found
# count() — how many times does substring appear?
print(s.count("Python")) # 2
print(s.count("is")) # 2
print("banana".count("an")) # 2
```

Method	Not Found Behaviour	Use When
find(sub)	Returns -1	You want to check IF it exists
index(sub)	Raises ValueError	You KNOW it exists, want position
count(sub)	Returns 0	You need frequency of substring

```
# Safe pattern using find()
s = "Python is great"
pos = s.find("great")
if pos != -1:
print(f"Found at index {pos}") # Found at index 10
else:
print("Not found")
```

5. Replace, Split & Join

The three most-used string operations in real projects

5.1 replace(old, new)

```
s = "I love Java, Java is great"
# Replace ALL occurrences by default
print(s.replace("Java", "Python")) # I love Python, Python is great
# Replace only first N occurrences
print(s.replace("Java", "Python", 1)) # I love Python, Java is great
# Delete a substring (replace with empty string)
print(s.replace("Java, ", "")) # I love Java is great
```

5.2 split(separator)

```
# Default split — splits on ANY whitespace
s = " one two three "
print(s.split()) # ['one', 'two', 'three'] — also strips spaces!
# Split on specific separator
s2 = "one,two,three,four"
print(s2.split(',')) # ['one', 'two', 'three', 'four']
# Limit splits with maxsplit
print(s2.split(',', 2)) # ['one', 'two', 'three,four']
# Split a sentence into words
sentence = "Python is awesome"
words = sentence.split() # ['Python', 'is', 'awesome']
```

5.3 join(iterable)

```
# join is the INVERSE of split
words = ['Python', 'is', 'awesome']
print(' '.join(words)) # Python is awesome
print('-'.join(words)) # Python-is-awesome
print(''.join(words)) # Pythonisawesome
# Common pattern: split → process → join
s = 'one two three'
words = s.split()
upper_words = [w.upper() for w in words] # (we will cover this later!)
result = ' '.join(upper_words)
print(result) # ONE TWO THREE
```

■ join() Tip

join() is called on the SEPARATOR string, not the list.

Correct: " ".join(words)

Wrong: words.join(" ") ← AttributeError

6. Check Methods (is... methods)

All return True or False

```
print("hello123".isalnum()) # True — all alphanumeric?
print("hello".isalpha()) # True — all letters?
print("12345".isdigit()) # True — all digits?
print(" ".isspace()) # True — all whitespace?
print("Hello World".istitle())# True — title case?
print("HELLO".isupper()) # True — all uppercase?
print("hello".islower()) # True — all lowercase?
# startswith / endswith
print("Python".startswith("Py")) # True
print("Python".endswith("on")) # True
print("Python".startswith(("Py","Go"))) # True ← accepts a tuple!
```

Method	Returns True When	Example
isalnum()	All chars are letters or digits	"abc123".isalnum()
isalpha()	All chars are letters	"hello".isalpha()
isdigit()	All chars are digits (0-9)	"123".isdigit()
isspace()	All chars are whitespace	" \t\n".isspace()
istitle()	Title-cased string	"Hello World".istitle()
isupper()	All letters are uppercase	"ABC".isupper()
islower()	All letters are lowercase	"abc".islower()
startswith(x)	Starts with prefix x	"Py".startswith("P")
endswith(x)	Ends with suffix x	"Py".endswith("y")

7. Padding & Alignment Methods

Formatting text into fixed-width fields

```
s = "Python"
print(s.center(20)) # " Python "
print(s.center(20, "-")) # "-------Python-------"
print(s.ljust(20, ".")) # "Python............."
print(s.rjust(20, ".")) # ".............Python"
# zfill — zero-pad number strings
print("42".zfill(5)) # "00042"
print("7".zfill(4)) # "0007"
# Real use — printing formatted table
for item, price in [("Apple",12),("Mango",25),("Banana",8)]:
print(item.ljust(10) + str(price).rjust(5))
# Apple 12
# Mango 25
# Banana 8
```

8. String Formatting

f-strings, .format(), and the % operator

8.1 f-Strings (Python 3.6+) — Recommended ■

```
name = "Rajeev"
score = 95.5
# Basic usage
print(f"Name: {name}") # Name: Rajeev
print(f"Score: {score}") # Score: 95.5
# Expressions inside {}
print(f"2 + 3 = {2 + 3}") # 2 + 3 = 5
print(f"{name.upper()} rocks!") # RAJEEV rocks!
# Number formatting
print(f"{score:.2f}") # 95.50 (2 decimal places)
print(f"{1000000:,}") # 1,000,000 (comma separator)
print(f"{score:.0f}%") # 96% (round to int)
# Multi-line f-string
card = f"""
Student : {name}
Score : {score:.1f}
Grade : {"A" if score >= 90 else "B"}
"""
print(card)
```

8.2 .format() Method

```
# Positional placeholders
print("Hello {}!".format("World")) # Hello World!
print("{0} + {1} = {2}".format(2, 3, 5)) # 2 + 3 = 5
# Named placeholders
print("{name} scored {score}".format(name="Raj", score=95))
# Raj scored 95
# Number formatting
print("{:.2f}".format(3.14159)) # 3.14
print("{:05d}".format(42)) # 00042
print("{:>10}".format("Hi")) # "        Hi" (right-align)
print("{:<10}".format("Hi")) # "Hi        " (left-align)
print("{:^10}".format("Hi")) # "    Hi    " (center)
```

8.3 % Operator (Legacy — for reading old code)

```
name = "Rajeev"
print("Hello %s" % name) # Hello Rajeev
print("Pi = %.2f" % 3.14159) # Pi = 3.14
print("%d items" % 5) # 5 items
print("%s scored %d" % ("Raj", 95)) # Raj scored 95
```

Style	Version	Recommended?	Key Advantage
f-string	3.6+	■ Yes	Readable, supports expressions
.format()	2.6+	■ Yes	Good for reusable templates
% operator	All	■■ Legacy	Seen in older codebases; avoid in new code

■ Use f-strings for all new code

f-strings are faster, more readable, and support inline expressions.

Learn .format() to read intermediate Python code.

Learn % only so you can understand legacy code — never use it in new projects.

9. Raw Strings

When backslashes should be taken literally

Prefix a string with **r** or **R** to make it a **raw string**. Backslashes are treated as literal characters — escape sequences are NOT processed. Essential for Windows file paths and regular expressions.

```
# Normal string — \n becomes a newline, \f becomes form-feed
print("C:\new\folder") # Oops! \n and \f are processed
# Raw string — backslash is just a backslash
print(r"C:\new\folder") # C:\new\folder ■
# File paths (Windows)
path = r"C:\Users\Rajeev\Desktop\project"
# Regular expressions (we will cover regex later)
import re
pattern = r"\d+\.\d+" # matches decimals like 3.14
m = re.search(pattern, "Value is 3.14")
print(m.group()) # 3.14
```

■■ Raw String Limitation

A raw string cannot end with an odd number of backslashes.

r"path\" → SyntaxError

Workaround: r"path" + "\\"

10. Method Quick Reference

All 20+ methods at a glance

Method	Returns	Description
upper()	str	All uppercase
lower()	str	All lowercase
title()	str	Title case
capitalize()	str	First char upper, rest lower
swapcase()	str	Flip every character's case
strip()	str	Remove whitespace/chars both sides
lstrip()	str	Remove from left only
rstrip()	str	Remove from right only
find(sub)	int	Index of first match, -1 if missing
index(sub)	int	Index of first match, ValueError if missing
count(sub)	int	Number of non-overlapping matches
replace(o,n)	str	Replace old with new
split(sep)	list	Split on separator, returns list
join(iter)	str	Join iterable with string as separator
startswith(x)	bool	Does string start with x?

endswith(x)	bool	Does string end with x?
isdigit()	bool	All chars are digits?
isalpha()	bool	All chars are letters?
isalnum()	bool	All chars are letters or digits?
isupper()	bool	All letters uppercase?
islower()	bool	All letters lowercase?
center(w,c)	str	Centre-align in field of width w
ljust(w,c)	str	Left-align
rjust(w,c)	str	Right-align
zfill(w)	str	Zero-pad to width w

11. Practice Exercises

30 problems covering all Part 2 topics

■ Beginner

Q1. Convert 'pymaster india' to uppercase, then to title case. Print both.

Q2. Remove spaces from ' clean this ' using strip(). Verify with len().

Q3. Check if the string '12345' is all digits. Check if 'hello123' is all alpha.

Q4. Replace 'bad' with 'good' in 'This is bad code with bad habits'.

Q5. Split 'one:two:three' on ':' and print the resulting list.

Q6. Join ['Python', 'is', 'fun'] with a space separator.

Q7. Check if 'Python' starts with 'Py' and ends with 'on'.

Q8. Print your name right-aligned in a field of width 30 using rjust().

Q9. Zero-pad the number string '7' to width 4.

Q10. Use an f-string to display: 'My name is X and my score is Y.2f'

■ Intermediate

Q11. Find the index of 'India' in 'PyMaster India'. Use both find() and index().

Q12. Count how many times 'an' appears in 'banana'. Count 'a' too.

Q13. Split the CSV string 'Rajeev,25,Varanasi,Python' into a list and print each value on a new line.

Q14. Use .format() to display a formatted invoice: Item, Quantity, Price in columns.

Q15. Strip all '*' characters from '***hello world***' using strip().

Q16. Check if user input (simulate with a variable) is a valid integer using isdigit().

Q17. Use swapcase() on 'PyMaster India' and explain what happened.

Q18. Using only find() (no 'in'), write a function that returns True if substring exists.

Q19. Split 'a::b::c' on '::' and rejoin with ' | '.

Q20. Format the number 1234567.89 with commas and 2 decimal places using f-string.

■ Think & Apply

Q21. Write a function that counts the number of vowels in a string using count() for each vowel.

Q22. Validate that a string is a proper title (each word starts with uppercase) using istitle().

Q23. Given a path string 'C:\\Users\\Rajeev', how would you store this using a raw string?

Q24. Write a function that removes all extra spaces from a string (split then join).

Q25. Given a CSV line of student data, split it and format a readable summary using f-strings.

Q26. Create a simple Caesar cipher: shift each letter in a word by 1 using ord() and chr().

Q27. Write a function title_case_words(s) that title-cases only words longer than 3 characters.

Q28. Given a list of names with inconsistent spacing and casing, normalize each name.

Q29. Use find() to find ALL occurrences of 'is' in a string (hint: use a loop with start param).

Q30. Build a simple receipt printer: given items and prices, print a formatted table using ljust/rjust.

■ Challenge Mode

For Q26-Q30, write the full working code — not just the answer.

These questions intentionally require combining multiple methods.

12. Quick Reference Cheat Sheet

Video #4.2 — Everything on one page

Case

```
s.upper() s.lower()
s.title() s.swapcase()
s.capitalize()
```

Check (is...)

```
s.isdigit() s.isalpha()
s.isupper() s.islower()
s.startswith(x) s.endswith(x)
```

Strip

```
s.strip() s.lstrip() s.rstrip()
s.strip("*") # specific char
```

Padding

```
s.center(20,"-")
s.ljust(20) s.rjust(20)
"7".zfill(4) # "0007"
```

Search

```
s.find(x) # -1 if absent
s.index(x) # ValueError if absent
s.count(x) # frequency
```

Formatting

```
f"Hi {name}"
"Hi {}".format(name)
f"{price:.2f}"
f"{n:,}" # comma sep
```

Replace / Split / Join

```
s.replace(old, new)
s.split(sep)
sep.join(list)
```

Raw Strings

```
r"C:\Users\Rajeev"
r"\d+\.\d+" # regex
```

codeWithShivah learn.rajeevshivah.me | PyMaster India — Video #4.2